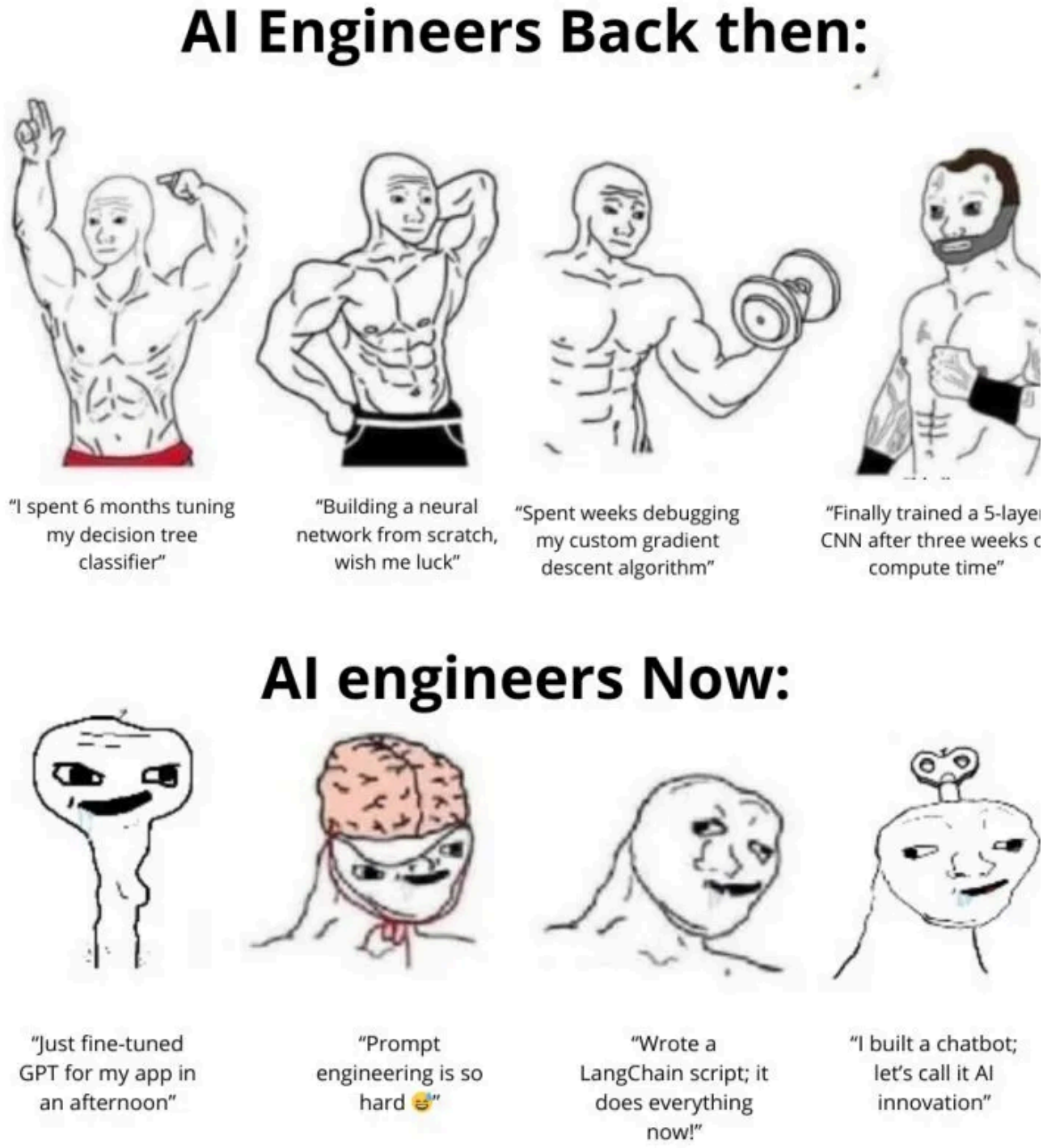
AI Engineer Roadmap 2026 [Ship or Die]

Here are the 6 + 1 Systems you need to build to get the $150k Architect role.

HIMANSHU RAMCHANDANI
FEB 15, 2026

44 3 9 Share



The game is over.

In 2023, you could get hired for knowing how to talk to ChatGPT.

In 2026, that skill will be worth $0.

A Microsoft recruiter told me last month, “we stopped reading resumes that list C Courses as experience.”

They don’t care about your certificate.

They care about one question - Can you ship?

There are 2 types of engineers right now:

1. Wrapper Dev: You call OpenAI’s API. You are dependent on them. You are ea replaced.
2. Systems Architect: You build local inference, orchestration, and agentic loop are unfireable.

If you want to be the second one, you need a portfolio.

Not a portfolio of Toy Projects.

A portfolio of Production Systems.

Here is the exact roadmap.

If you build these, you are in the top 1% of talent globally.

Before I get into the artifacts, I need to address the biggest gap I see in every oth roadmap out there - **they skip the foundations entirely.**

Engineers who skip foundations can ship.

But when something breaks in production, they are guessing.

Engineers who understand what is happening inside the model are diagnosing.

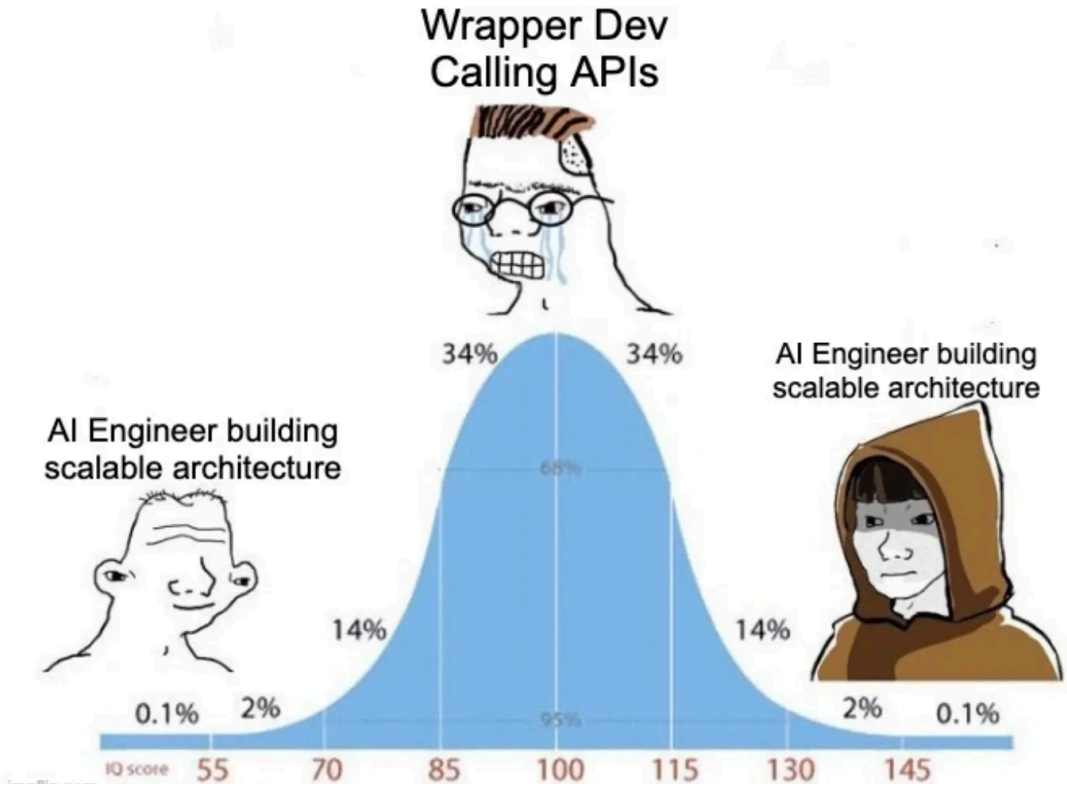
That difference is worth $100k in salary.

So we start there.

Do This Before Everything Else

Most roadmaps skip this.

That is why most engineers plateau at the wrapper level.



You do not need to become a researcher.

But you need to build a transformer from scratch, build a tokenizer from scratch, understand what autograd is doing.

Not because you will use these implementations in production.

Because once you have built them, you will never be lost again.

You will know exactly where to look when a model is failing.

This is for building skin in the game, and you also know this from inside thats wh are reading this roadmap.

What you build first

Build these 3 things:

1. **Build Micrograd**

It is Andrej Karpathy's implementation of a scalar-valued autograd engine in rou 100 lines of Python.

- No PyTorch
- No abstractions

You implement forward and backward passes yourself.

After this, the concept of a gradient will never feel abstract again, and fine-tunin hyperparameters will start to make intuitive sense.

The repo is at github.com/karpathy/micrograd

2. **Build a BPE tokenizer**

Byte Pair Encoding is the algorithm behind most modern LLM tokenizers.

Build it from scratch using the minbpe repo at github.com/karpathy/minbpe

Takes about 3 hours.

After this, you will understand why certain domain-specific terms get split into w subword tokens and why that matters enormously for fine-tuning on specialized like medical or legal text.

3. **Build a small transformer**

Work through Karpathy's nanoGPT repo and reproduce a character-level GPT.

You are implementing multi-head self-attention, positional embeddings, and laye normalization by hand.

After this, when someone says “the model has a 128k context window,” you will k exactly what that means for memory and compute.

Read 3 research papers before moving on.

1. **How to read a Paper** - this paper will show you the right way to read a resea paper. (thats crazy - a research paper on how to read a research paper)
2. **Attention is All You Need** (2017) - it is the architecture every model in this roadmap is built on
3. **Language Models are Few-Shot Learners** (2020), which is the GPT-3 pap explains why prompting works at all.

What you learn

You will learn how neural networks store weights as 32-bit floating point values b default

- why that matters for memory
- what a forward pass actually computes
- what a backward pass actually does when it propagates gradients
- why cross-entropy loss is the right loss function for language modeling
- why tokenization is the first thing to check when a model performs poorly or domain-specific text
- why attention is $O(n^2)$ in sequence length, which is the root cause of every c window limitation you will ever hit

6 + 1 Artifacts That Prove You Can Build

Forget roadmaps that list 47 topics to study.

This is a roadmap of things to build and ship.

Each one teaches you a different part of the AI engineering stack.

Each one becomes a portfolio piece that makes recruiters want to interview you.

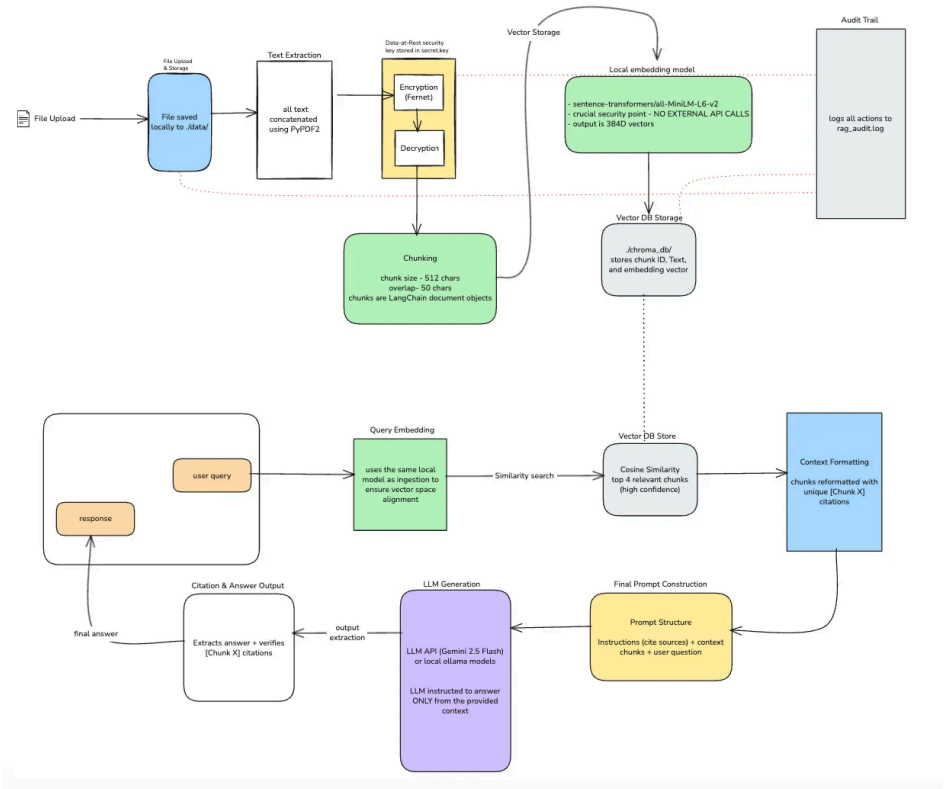
How Each Artifact Is Structured

Every artifact follows the same pattern.

- **Problem we are solving**
- **What you build**
- **Why this matters -** why companies care about this skill?
- **What you actually learn (Contextual Curriculum) -** Think of this as your n roadmap for each project. You only learn what you need to build the solutior
- **Research Papers**
- **Tech stack**
- **Proof of work**

Let's go.....

[Artifact 1] Offline-First RAG System with Quantized SLMs



Problem we are solving

Lawyers cannot upload case files to ChatGPT (HIPAA violations).

Doctors cannot use AI for patient records.

Privacy regulations block them.

Anyone with sensitive documents is stuck choosing between privacy and produc

What you build

A desktop app that lets people chat with sensitive documents completely offline

No data leaves the device.

AI for private files, but nothing goes to the cloud.

Why this matters

Most people think AI requires massive servers. Which is not right.

You can run language models on a normal laptop if you know how to optimize th

This project teaches you that skill.

Companies building on-device AI, healthcare apps, or legal tech are desperate f engineers who understand this.

What you actually learn (Contextual Curriculum)

Module 1 - Quantization fundamentals

- how neural networks store weights (32-bit floating point by default)
- quantization methods (GGUF, GPTQ, AWQ)
- converting models to 4-bit without destroying quality
- memory calculations (how much RAM you actually need)

Module 2 - Running LLMs locally

- llama.cpp architecture and C++ bindings
- cpu inference optimization
- model loading strategies (lazy loading vs full load)
- handling out-of-memory errors gracefully

Module 3 - RAG pipeline basics

- document chunking strategies (semantic vs fixed-size)
- text embedding models (all-MiniLM, BGE, nomic-embed)
- vector databases for local storage (ChromaDB, FAISS)
- similarity search algorithms (cosine similarity, dot product)

Module 4 - Building the retrieval system

- embedding documents locally
- query-time retrieval logic

- context window management (fitting retrieved chunks into prompts)
- re-ranking results for better accuracy

Module 5 - Desktop app development

- streamlit basics for quick UIs
- file upload handling
- real-time streaming responses
- packaging Python apps for distribution

Research papers to read:

1. **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks** - original RAG paper
2. **Precise Zero-Shot Dense Retrieval without Relevance Labels**

Tech stack

llama.cpp, ChromaDB, Streamlit, Python, Sentence Transformers

Proof of work (what to showcase)

Deploy the desktop app.

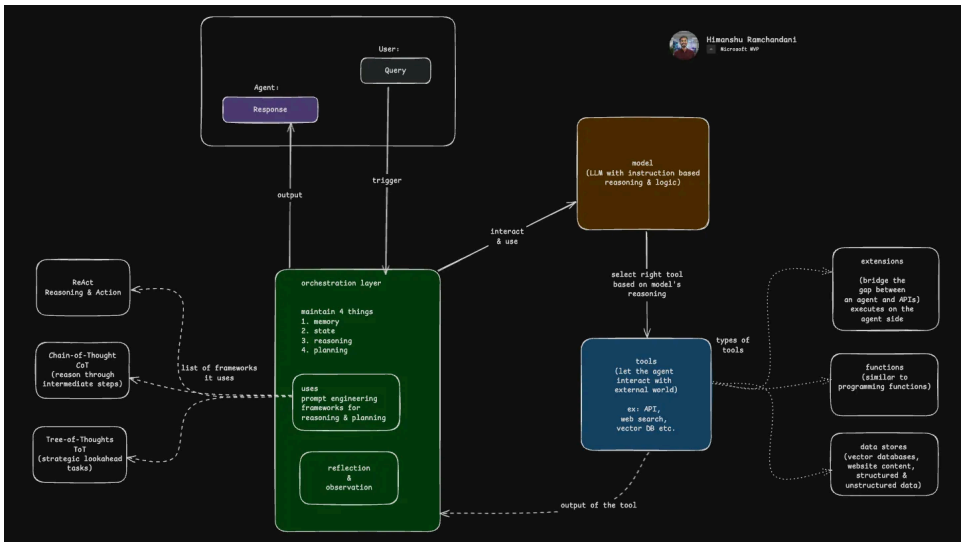
Get 10 beta users (lawyers, doctors, anyone with private docs).

Record a demo video showing:

- upload a sensitive document
- ask complex questions
- get accurate answers
- prove nothing left the device (disconnect wifi mid-conversation)

Write a blog post - I Built ChatGPT for Lawyers That Runs Offline

[Artifact 2] Self-Healing Coding Agent with Stateful Agentic Loops



Problem we are solving

Developers waste hours debugging.

Most AI coding tools generate code but do not fix their own mistakes.

They produce broken code, you fix it manually, and the cycle repeats.

What if an AI could write code, run it, read error logs, and fix its own bugs until te pass?

What you build

A command-line tool that writes code, executes it in a sandbox, reads error mes and iterates until all tests pass.

Like an intern that debugs itself.

You give it a task, walk away, come back to working code.

Why this matters

Agentic AI is the hottest area in 2026.

Everyone wants agents.

Almost nobody knows how to build them properly.

Most tutorials show toy examples with no error handling.

This project teaches you production-grade agent architecture, which includes st management, sandboxed execution, and self-correction loops.

What you actually learn (Contextual Curriculum)

Module 1 - Agent fundamentals

- what makes something an agent vs a simple script
- reAct pattern (Reasoning + Acting)
- planning vs reactive agents
- when to use which architecture

Module 2 - State management in agents

- stateful vs stateless loops
- langGraph for building cyclic workflows
- memory persistence across iterations
- tracking previous attempts and failures

Module 3 - Safe code execution

- docker basics (containers, images, volumes)
- docker SDK for Python
- spinning up containers programmatically
- cleaning up resources after execution

Module 4 - Code validation

- abstract Syntax Trees (AST) in Python
- parsing code before running it
- detecting syntax errors statically
- security checks (blocking dangerous operations)

Module 5 - Reflection and self-correction

- prompt engineering for error analysis
- structured outputs from LLMs (JSON mode)
- extracting actionable fixes from error messages
- loop termination conditions (max attempts, success criteria)

Module 6 - Testing and verification

- pytest basics
- writing tests that agents can understand
- automated test execution
- verifying code correctness programmatically

Research Paper to Read:

ReAct - Synergizing Reasoning and Acting in Language Models

Tech stack

LangGraph, Docker SDK, PyTest, AST, Gemini API, Python.

Proof of work (what to showcase)

Record a video of your agent fixing a real bug. Show:

- give it a broken Python function
- watch it run the code, fail, read the error

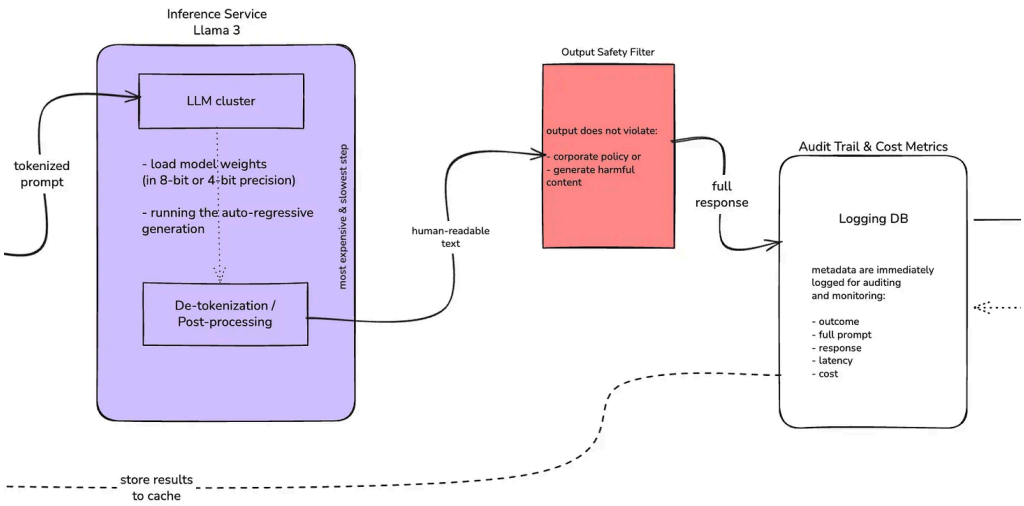
- see it modify the code and try again
- watch it succeed after 2-3 attempts

Write a technical blog post breaking down the architecture.

Include the LangGraph diagram, code snippets for the reflection prompt, and examples of bugs it fixed.

Post content on - I built an AI that debugs itself.

[Artifact 3] Real-Time Low-Latency Full-Duplex Audio Streaming Server



Problem we are solving

Voice AI sounds robotic because most implementations are turn-based.

You talk, it waits, then it responds.

Real humans interrupt each other mid-sentence.

Current voice bots cannot handle this.

They feel unnatural and frustrating to use.

What you build

A voice AI server that handles real conversations with interruptions.

Full-duplex means both sides can talk simultaneously.

The AI detects when you interrupt, stops talking, listens, and responds naturally.

Under 500ms latency.

Feels like talking to a human.

Why this matters

Voice AI is exploding in customer service, healthcare, and sales.

But most implementations fail the natural conversation test.

Full-duplex with low latency is hard.

Companies building voice agents, call center automation, or conversational AI wi
top dollar for engineers who can build this.

What you actually learn (Contextual Curriculum)

Module 1 - Real-time communication fundamentals

- http vs WebSockets (why HTTP fails for real-time)
- websocket protocol basics
- connection lifecycle (connect, message, disconnect)
- handling reconnections and network failures

Module 2 - Audio streaming basics

- audio formats (WAV, MP3, PCM)
- sample rates and bit depth
- chunking audio for streaming (chunk size tradeoffs)
- buffering strategies

Module 3 - Speech-to-text integration

- streaming STT vs batch processing
- deepgram streaming API
- handling partial transcripts
- end-of-speech detection

Module 4 - Voice Activity Detection (VAD)

- silero VAD model
- detecting speech vs silence in audio streams
- detecting interruptions
- calibrating sensitivity thresholds

Module 5 - LLM streaming

- openAI Realtime API
- server-sent events (SSE)
- streaming tokens as they generate
- stopping generation mid-stream when interrupted

Module 6 - Text-to-speech

- elevenLabs streaming API
- latency optimization techniques
- voice cloning basics

- handling pronunciation edge cases

Module 7 - Concurrency and async programming

- python asyncio fundamentals
- managing multiple concurrent streams
- non-blocking I/O
- race conditions and deadlocks

Module 8 - Latency optimization

- measuring end-to-end latency
- profiling bottlenecks
- parallel processing where possible
- caching strategies

Tech stack

FastAPI, WebSockets, Silero VAD, Deepgram, OpenAI Realtime API, ElevenLabs, Python asyncio.

Proof of work (what to showcase)

Deploy it live.

Give friends the URL to call it.

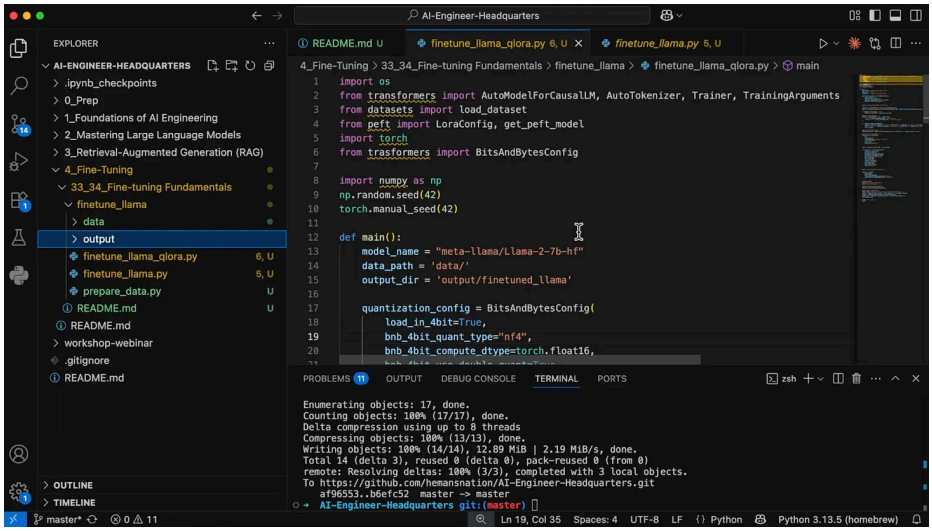
Record the best conversations showing:

- natural interruptions handled smoothly
- sub-500ms response times
- realistic voice quality
- complex multi-turn dialogue

Write a technical deep-dive - How I Built a Voice AI That Feels Human

Include architecture diagrams showing the WebSocket flow, latency measureme and code for handling interruptions.

[Artifact 4] Synthetic Data Generation & Fine-Tuning Specialist



Problem we are solving

GPT-4 costs $30 per million tokens.

For high-volume tasks like generating SQL queries, writing product descriptions, answering support tickets, these costs destroy budgets.

Meanwhile, GPT-4 is overkill for most tasks.

A small 8B model fine-tuned for one specific job can match GPT-4 quality at 1% cost.

What you build

A pipeline that generates synthetic training data using GPT-4, then fine-tunes a 8B model to outperform GPT-4 on a specific niche task.

A model that writes SQL queries better than GPT-4 for e-commerce databases.

Include automated evaluation to prove it works.

Why this matters

Most companies cannot afford GPT-4 at scale.

They need custom models.

But hiring a team to label 50,000 training examples costs $200,000 and takes m

Synthetic data generation solves this.

You use a teacher model (GPT-4) to create training data automatically.

Then you fine-tune a cheap model.

Companies with custom AI needs will hire you immediately if you can do this.

What you actually learn (Contextual Curriculum)

Module 1 - Synthetic data generation

- prompting strategies for data generation
- few-shot examples for consistent outputs
- quality control (detecting bad synthetic examples)
- scaling generation (batching, parallelization)

Module 2 - Dataset curation

- deduplication techniques
- filtering low-quality examples
- balancing dataset distribution
- train/validation/test splits

Module 3 - Instruction tuning format

- conversation templates
- system prompts vs user prompts
- data formatting for different model families (Llama, Mistral, Phi)
- tokenization and special tokens

Module 4 - Fine-tuning fundamentals

- full fine-tuning vs LoRA
- how LoRA works (low-rank adaptation matrices)
- qLoRA (quantized LoRA for 4-bit training)
- hyperparameter selection (learning rate, batch size, epochs)

Module 5 - Training infrastructure

- gpu selection and cost optimization
- using cloud GPUs (RunPod, Lambda Labs)
- monitoring training (loss curves, gradient norms)
- checkpointing and recovery

Module 6 - Evaluation methods

- traditional metrics (BLEU, ROUGE, exact match)
- llm-as-a-Judge evaluation
- building automated benchmarks
- human evaluation sampling

Module 7 - Deployment

- exporting models to GGUF
- model quantization for inference

- hosting options (Hugging Face, Replicate, local)
- api wrapping with FastAPI

Research Paper to Read:

1. **LoRA - Low-Rank Adaptation of LLMs**
2. **Direct Preference Optimization - Your Language Model is Secretly a Re Model**

Tech stack

Unsloth, Hugging Face TRL, WandB, Ragas, GPT-4 API, Python

Proof of work (what to showcase)

Fine-tune a model on a real task (SQL generation, email writing, data extraction)

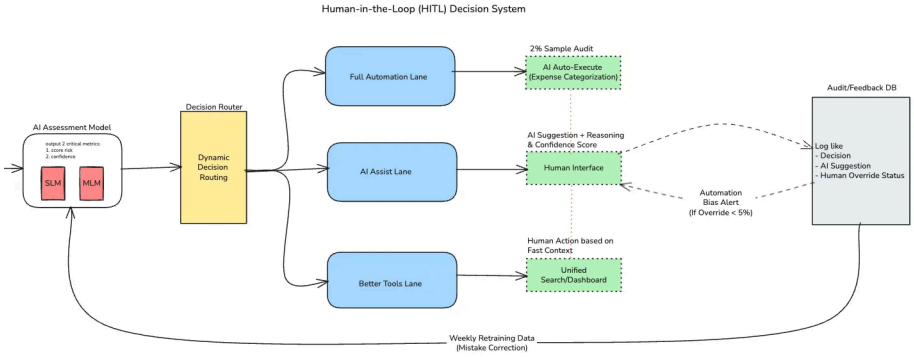
Run benchmarks comparing it to GPT-4. Create graphs showing:

- accuracy comparison
- cost per 1M tokens comparison
- latency comparison

Write a blog post with the results - How I Built a Model That Beats GPT-4 at SQL 1% of the Cost

Include the training code, dataset generation prompts, and evaluation methodol

[Artifact 5] Event-Driven Workflow Orchestrati & Human-in-the-Loop System



Problem we are solving

Customer support tickets pile up.

Humans cannot respond fast enough.

But fully autonomous AI sends wrong answers and damages customer trust.

Companies need AI that does 80% of the work and asks humans to approve the 20%.

This is the human-in-the-loop problem that every enterprise faces.

What you build

A backend system that listens to Slack or Jira events, triggers multi-agent workf and pauses for human approval before taking action.

When a support ticket arrives, agents research the issue, draft a response, and s to a human for approval via Slack.

If approved, the response gets sent automatically.

Why this matters

Real companies do not want fully autonomous AI.

They want AI that augments humans, not replaces them.

Human-in-the-loop is the business model.

This project teaches you event-driven architecture, workflow orchestration, and persistence.

What you actually learn (Contextual Curriculum)

Module 1 - Event-driven architecture fundamentals

- events vs requests (when to use which)
- webhooks basics
- setting up webhook endpoints with FastAPI
- securing webhooks (signature verification)

Module 2 - Message queues

- why queues matter (decoupling, reliability)
- redis for lightweight queuing
- pub/sub patterns
- handling backpressure

Module 3 - Multi-agent orchestration

- designing agent teams (researcher, writer, reviewer)
- task routing logic
- inter-agent communication
- handoffs and delegation

Module 4 - State persistence

- why agents need databases
- postgreSQL basics
- storing conversation history

- resuming workflows after crashes or restarts

Module 5 - LangGraph for workflows

- building stateful graphs
- conditional edges (if-then routing)
- loops and cycles
- human-in-the-loop nodes (pausing execution)

Module 6 - Integration with external tools

- slack API (reading messages, sending notifications)
- jira API (creating tickets, updating status)
- oauth flows for user authentication
- rate limiting and retries

Module 7 - Monitoring and observability

- logging workflows with LangSmith
- tracing agent decisions
- debugging failures in production
- alerting when agents get stuck

Tech stack

LangGraph, PostgreSQL, Redis, Webhooks, Slack API, FastAPI, Python

Proof of work (what to showcase)

Deploy it internally at a company or for a side project.

Track metrics:

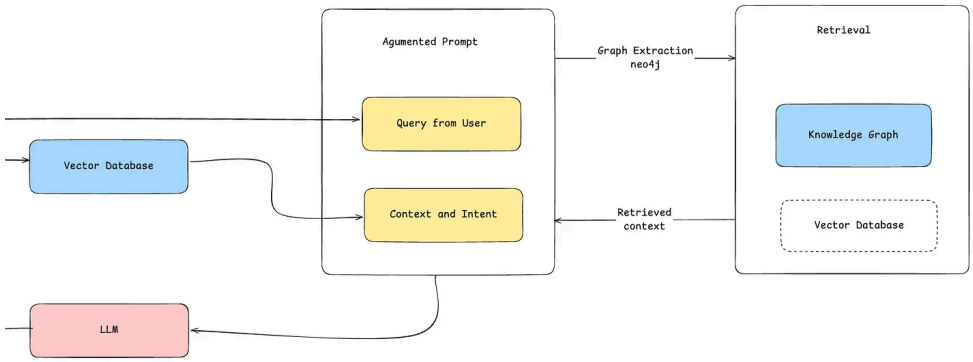
- time saved per ticket

- approval rate (how often humans approve vs reject)
- resolution time (before vs after)

Write a case study - How I Built an AI Support Agent That Needs Human Approv

Include architecture diagrams showing the event flow, screenshots of Slack appı and code for the state machine.

[Artifact 6] Hybrid Retrieval Memory Systems (Vector + Graph)



Problem we are solving

AI assistants forget context.

They do not remember who you talked about last week, what projects you are w on, or relationships between people and ideas.

Vector search finds similar content but misses connections.

Graph search finds relationships but misses semantic meaning.

Most engineers only know one.

You need both.

What you build

A digital twin that remembers everything about you using both vector and graph memory.

It recalls past conversations, understands relationships between people and proj and writes emails in your style.

Think of it as an AI clone of yourself that gets smarter over time.

Why this matters

Personalized AI is the next frontier.

Companies building personal assistants, CRM tools, or knowledge management platforms need engineers who understand advanced retrieval.

Combining vector and graph databases is rare.

This skill makes you extremely valuable.

What you actually learn (Contextual Curriculum)

Module 1 - Vector databases deep dive

- embeddings review (how text becomes vectors)
- HNSW algorithm (approximate nearest neighbors)
- index types and tradeoffs
- metadata filtering

Module 2 - Graph databases fundamentals

- nodes, edges, and properties
- neo4j Cypher query language
- modeling relationships (entities, events, concepts)
- traversal algorithms (shortest path, PageRank)

Module 3 - Knowledge graph construction

- entity extraction from text (NER)
- relationship extraction
- coreference resolution (knowing “he” refers to “John”)
- building graphs programmatically

Module 4 - Hybrid search strategies

- when to use vector vs graph vs keyword search
- combining results from multiple systems
- scoring and ranking hybrid results
- query routing logic

Module 5 - Memory compression

- recursive summarization
- forgetting strategies (what to keep vs discard)
- context window management
- long-term vs short-term memory

Module 6 - Style transfer and personalization

- few-shot prompting for writing style
- extracting stylistic features from text
- voice and tone consistency
- testing personalization quality

Module 7 - Privacy and encryption

- end-to-end encryption basics
- storing sensitive data securely

- key management
- compliance considerations (GDPR)

Research Paper to Read:

RAPTOR paper (2024)

Tech stack

Neo4j, NetworkX, LangChain, Sentence Transformers, Cryptography libraries, Py

Proof of work (what to showcase)

Use the system for two weeks.

Document:

- examples of it recalling past conversations accurately
- emails it wrote in your style
- relationships it remembered (people, projects, context)

Write a blog post - I Built an AI That Remembers Everything About Me

Include before/after examples of AI-generated emails, architecture diagrams sho vector + graph integration, and code for the hybrid search logic.

> I am adding a new artifact to gain evaluation and monitoring skills, which are r important in real-world AI software development.

[Artifact 7] Production Eval Harness and Quality Monitoring System

Most engineers build a system, demo it, and move on.

Senior engineers build the system that watches the system.

Problem

AI systems degrade silently.

A new model version ships and certain edge cases quietly break.

Retrieval quality drifts as your document corpus grows.

Your fine-tuned model performs worse on the newest customer queries because use terminology from a product launched after your training cutoff.

Without an eval harness running continuously, you find out when a customer twe about it.

What you build

A continuous evaluation pipeline that monitors a deployed AI system, detects qu degradation, and alerts before users notice.

You define a benchmark, run it on a schedule against your production system, tr metrics over time, and build a dashboard that shows quality trends.

When a metric drops below a threshold, it pages you.

What you actually learn (Contextual Curriculum)

Module 1 - covers eval design

- writing deterministic evals with ground truth answers
- writing non-deterministic evals using LLM-as-a-Judge with structured scori rubrics
- designing eval sets that cover edge cases, not just common cases

Module 2 - covers retrieval evals specifically

- precision, recall, and MRR for measuring retrieval quality

- context relevance scoring
- faithfulness scoring
- ragas automates most of this

Module 3 - covers generation evals

- exact match for structured outputs like SQL or JSON
- semantic similarity using embedding cosine similarity for free-text outputs
- LLM-as-a-Judge with a rubric

Module 4 - covers regression testing pipelines

- full eval suite against a new model version before it ships
- compare the results to a baseline
- flag regressions automatically

This is the same pattern software engineers use for unit tests, applied to AI quali

Module 5 - covers statistical significance

- bootstrap confidence intervals or a paired t-test to know whether the differe real

Module 6 - covers drift detection

- embedding drift
- maximum Mean Discrepancy
- set up alerts when drift exceeds a threshold

Module 7 - covers building the dashboard.

- A simple FastAPI backend with a PostgreSQL store for eval run history and a Streamlit or React frontend showing quality trends over time.
- Just numbers you can look at every morning.

Tech stack

Ragas, LangSmith, PostgreSQL, FastAPI, Python, Streamlit

Proof of work

Build this on top of any previous artifact.

Run it for two weeks

Show your quality trend charts

Show one regression you caught before it hit users and how you fixed it.

Write a blog post: How I Built a System That Watches My AI

What Gets You Hired

Recruiters are not reading your resume, looking for frameworks you know.

They are looking for two things

- proof you shipped something real
- evidence you know how it works when it breaks

The 6 artifacts + 1 above address both.

Every one of them has a measurable proof of work.

Every one of them builds the diagnostic intuition you need to debug production failures.

Build all seven and you have a portfolio that most engineers with five years of experience cannot match.

You have the roadmap.

You have the list of artifacts.

You have the tech stack.

You have two choices:

Choice 1 - Solo Grind

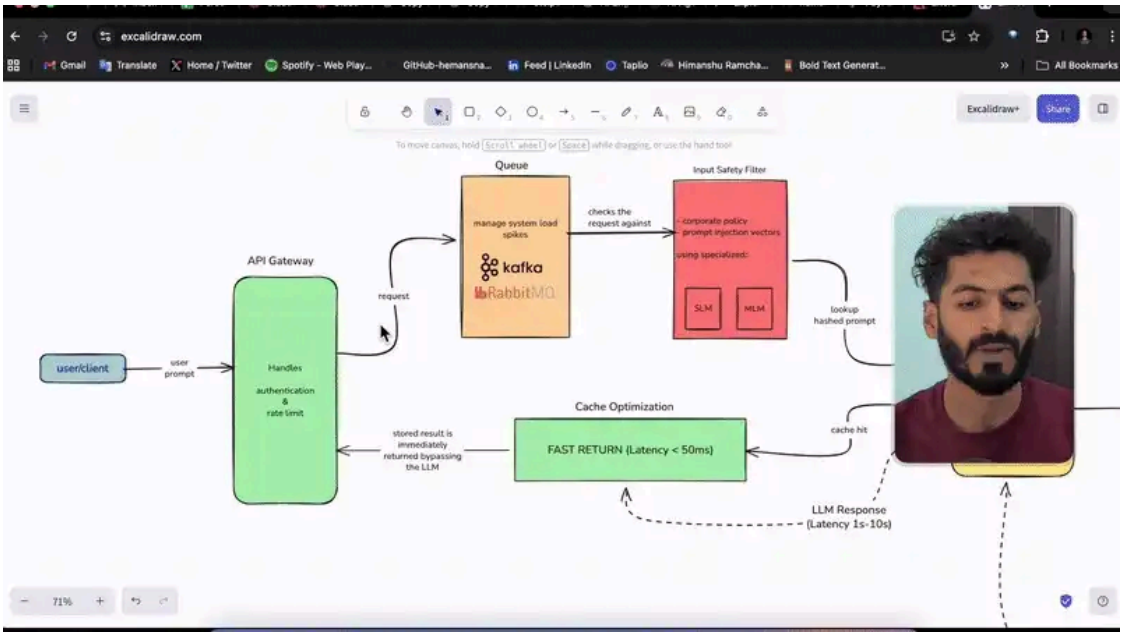
You can spend the next 6 months trying to figure this out alone.

You will get stuck on Docker errors.

You will get confused by LangGraph documentation.

You will likely quit after Artifact 2.

Choice 2 - AI Engineer HQ Accelerator



You join me and 50 other builders.

We build these 6 Artifacts in 8 Weeks.

You leave with a deployed portfolio that makes recruiters drool.

I am not selling information. Information is free.

I am selling **Execution**.

Cohort 2 registrations will start soon.

If you want to stop building wrappers and start building systems...

[**CLICK HERE TO SECURE YOUR SPOT**]

NOTE: I provide a **Regional Parity Grant** on the price. Please DM me your count and I will share the parity link with you with a grant upto 65%.

Let's build.

44 Likes · 9 Restacks

Previous Ne:

Discussion about this post

Comments Restacks

Write a comment...

Himanshu Ramchandani Feb 15
Author
Join the Cohort 1 - Starting on 14th Feb, 2026
AI Engineer HQ Accelerator: https://www.masterdexter.io/ai-engineer-hq
LIKE (1) REPLY

Vipulkumar Patel Mar 4
Hey Himanshu let me know if you have any open slot for cohort 1. I am interested.
LIKE (1) REPLY

1 reply by Himanshu Ramchandani

1 more comment...

© 2026 Himanshu Ramchandani · Privacy · Terms · Collection notice
Substack is the home for great culture